

Atividade Prática A1 – Grafos

INE5413 – Ciências da Computação – UFSC

Integrantes: Nome 1, Nome 2, Nome 3

Item 1 – Representação do grafo. Foi implementada a classe `Graph` para representar um grafo não-dirigido e ponderado. A estrutura principal adotada foi uma lista 1-indexada de dicionários de adjacência, na qual `adj[u][v]` armazena diretamente o peso da aresta $\{u, v\}$. Essa escolha permite tempo $O(1)$ médio para as operações `haAresta(u, v)`, `peso(u, v)`, `grau(v)` e `rotulo(v)`, atendendo ao requisito do enunciado sempre que possível. Os rótulos dos vértices foram armazenados em uma lista 1-indexada separada, também com acesso $O(1)$. A leitura do arquivo é linear no tamanho da entrada, isto é, $O(|V| + |E|)$.

Item 2 – Busca em largura (BFS). Para a BFS foram utilizados um vetor booleano de visitados, um vetor de níveis e uma fila `deque`. A fila garante inserção e remoção no início/fim em $O(1)$, tornando a busca global $O(|V| + |E|)$. Como o exercício exige a saída agrupada por nível, os vértices descobertos foram adicionados em uma lista de listas, preservando uma organização simples para impressão posterior. Os pesos do arquivo são ignorados, conforme solicitado.

Item 3 – Ciclo euleriano. A solução primeiro verifica duas condições necessárias e suficientes para grafos não-dirigidos: todos os vértices de grau não nulo devem pertencer ao mesmo componente conexo e todos os graus devem ser pares. Caso essas condições sejam satisfeitas, o ciclo é construído com o algoritmo de Hierholzer, que percorre cada aresta essencialmente uma vez, com custo $O(|V| + |E|)$. Foi utilizada uma estrutura auxiliar de multiplicidade de arestas para consumir as arestas durante a construção do ciclo sem alterar a representação original do grafo.

Item 4 – Menores caminhos de fonte única. Foi escolhido o algoritmo de Dijkstra, implementado com fila de prioridade baseada em `heapq`. Essa estratégia é adequada para pesos não negativos e apresenta complexidade $O((|V| + |E|) \log |V|)$. As distâncias mínimas são guardadas em um vetor `dist` e os predecessores em um vetor `parent`, permitindo reconstruir o caminho mínimo até cada destino exigido pelo enunciado. A impressão final mostra, para cada vértice, o caminho completo a partir da origem e a respectiva distância acumulada.

Item 5 – Floyd-Warshall. Para os menores caminhos entre todos os pares, foi adotada uma matriz de distâncias `dist[i][j]`, inicializada com infinito, zero na diagonal principal e peso das arestas nas posições correspondentes. O algoritmo de Floyd-Warshall foi então aplicado com três laços aninhados, resultando em complexidade $O(|V|^3)$ e uso de memória $O(|V|^2)$. Apesar de mais custoso que Dijkstra para uma única origem, ele é apropriado quando se deseja obter todas as distâncias entre todos os pares de vértices em uma única execução.

Validação. Antes da execução em ambiente fechado, foram construídos testes unitários com `unittest` para validar a leitura do grafo, a BFS, a detecção/construção de ciclo euleriano, o Dijkstra e o Floyd-Warshall. Dessa forma, a solução foi verificada automaticamente em cenários pequenos e controlados, reduzindo a chance de erro funcional na entrega.